P1021R6
Mike Spertus, Symantec
mike_spertus@symantec.com
Timur Doumler
papers@timur.audio
Richard Smith
richardsmith@google.com
2022-05-15
Audience: Core Working Group

Filling holes in Class Template Argument Deduction

This paper proposes filling several gaps in Class Template Argument Deduction.

Document revision history

R0, 2018-05-07: Initial version
R1, 2018-10-07: Refocused paper on filling holes in CTAD
R2, 2018-11-26: Following EWG guidance, removed proposal for CTAD from partial template argument lists
R3, 2019-01-21: Added wording and change target to Core Working Group
R4, 2019-06-17: Updated wording in response to CWG comments
R5, 2019-08-05: Updated algorithm for aggregates; moved wording to separate wording papers
R6, 2022-05-15: Updated references to wording papers

Rationale

As one of us (Timur) has noted when giving public presentations on using class template argument deduction, there are a significant number of cases where it cannot be used. This always deflates the positive feelings from the rest of the talk because it is accurately regarded as artificially inconsistent. In particular, listeners are invariably surprised that it does not work with aggregate templates, type aliases, and inherited constructors. We will show in this paper that these limitations can be safely removed. Note that some of these items were intentionally deferred from C++17 with the intent of adding them in C++20.

Class Template Argument Deduction for aggregates

We propose that Class Template Argument Deduction works for aggregate initialization as one shouldn't have to choose between having aggregates and deduction. This is well illustrated by the following example:

C++17	Proposed

`template<class T>`
`struct Point { T x; T y; };`
 	
`// Aggregate: Cannot deduce`
`Point<double> p{3.0, 4.0};`
`Point<double> p2{.x = 3.0, .y = 4.0};`
	`template<class T>`
	`struct Point { T x; T y; };`
	
	`// Proposed: Aggregates deduce`
	`Point p{3.0, 4.0};`
	`Point p2{.x = 3.0, .y = 4.0};`

For the code on the right hand side to work in C++17, the user would have to write an explicit deduction guide. We believe that this is unnecessary and error-prone boilerplate and that the necessary deduction guide should be implicitly synthesized by the compiler from the arguments of a braced or designated initializer during aggregate initialization.

Algorithm

In the current Working Draft, an aggregate class is defined as a class with no user-declared or inherited constructors, no private or protected non-static data members, no virtual functions, and no virtual, private or protected base classes. While we would like to generate an aggregate deduction guide for class templates that comply with these rules, we first need to consider the case where there is a dependent base class that may have virtual functions or virtual base classes, which would violate the rules for aggregates. Fortunately, that case does not cause a problem because any deduction guides that require one or more arguments will result in a compile-time error after instantiation, and non-aggregate classes without user-declared or inherited constructors generate a zero-argument deduction guide anyway. Based on the above, we can safely generate an aggregate deduction guide for class templates that comply with aggregate rules.

When P0960R0 was discussed in Rapperswil, it was voted that in order to allow aggregate initialization from a parenthesized list of arguments, aggregate initialization should proceed as if there was a synthesized constructor. We can use a similar approach to also synthesize the required additional deduction guide during class template argument deduction as follows:

1. Given a primary class template `C`, determine whether it satisfies all the conditions for an aggregate class ([dcl.init.aggr]/1.1 - 1.4), except that we are not considering the possibility of dependent base classes violating those conditions.
2. If yes, let `x_1`, ..., `x_n` be the elements of the initializer list.
3. Consider the elements ([dcl.init.aggr]/2) of the aggregate (which may or may not depend on its template arguments). For each initializer `x_i`, find the element `e_i` that this initializer would be initializing, according to the rules of aggregate initialization.
4. Form a hypothetical constructor `C(T_1, ..., T_N)`, where `T_i` is declared type of the element `x_i`.
5. For the set of functions and function templates formed for [over.match.class.deduct], add an additional function template derived from this hypothetical constructor as described in [over.match.class.deduct]/1.

Brace elision

There is a slight complication resulting from subaggregates, and the fact that nested braces can be omitted when instantiating them:

```
struct Foo { int x, y; };
struct Bar { Foo f; int z; };
Bar bar{1, 2};   // Initializes bar.f.x and bar.f.y to 1; zero-initializes bar.z
```

In this case, we have two initializers, but they do not map to the two elements of the aggregate type `Bar`, instead initializing the sub-elements of the first subaggregate element of type `Foo`. This in itself is not a problem, as we can still easily determine which element (or sub-element) is initialized by which initializer. However it becomes a bigger problem if one of the elements is dependent on a template parameter, and we cannot tell during CTAD whether it is a subaggregate or not (or how many sub-elements it holds:

```
template <typename T>
struct Bar { T f; int z; };  // T might be a subaggregate!
```

We therefore propose to avoid these problems by not considering brace elision for dependent types during class template argument deduction. Therefore, for example, `Bar bar{1.0f, 2}` deduces `Bar<float>`, which is exactly what the user would expect.

Deduction guide depends on initializer

Another interesting property of this algorithm is that the new deduction guide depends on the initializer, and might be different for each one. Therefore, even for the same class template, the set of deduction candidates is different depending on where in the code CTAD is performed. However, this is not novel. We already have several such situations in C++17: an explicit deduction guide can be declared later in the code, adding a new candidate to the set. As another example, it is possible to declare a primary template and then define it later. In this situation, the set of deduction candidates will be different before and after that definition.

Class Template Argument Deduction for alias templates

While Class Template Argument Deduction makes type inferencing easier when constructing classes, it doesn't work for type aliases, penalizing the use of type aliases and creating unnecessary inconsistency.We propose allowing Class Template Argument Deduction for type aliases as in the following example.

vector	pmr::vector (C++17)	pmr::vector (proposed)
`vector v = {1, 2, 3};` `vector v2(cont.begin(), cont.end());`	`pmr::vector<int> v = {1, 2, 3};` `pmr::vector<decltype(cont)::value_type> v2(cont.begin(), cont.end());`	`pmr::vector v = {1, 2, 3};` `pmr::vector v2(cont.begin(), cont.e`

`pmr::vector` also serves to illustrate another interesting case. While one might be tempted to write

```
pmr::vector pv({1, 2, 3}, mem_res); // Ill-formed (C++17 and proposed)
```

this example is ill-formed by the normal rules of template argument deduction because `pmr::memory_resource` fails to deduce `pmr::polymorphic_allocator<int>` in the second argument.

While this is to be expected, one suspects that had class template argument deduction for alias templates been around when `pmr::vector` was being designed, the committee would have considered allowing such an inference as safe and useful in this context. If that was desired, it could easily have been achieved by indicating that the `pmr::allocator` template parameter should be considered non-deducible:

```
namespace pmr {
  template<typename T>
  using vector = std::vector<T, type_identity_t<pmr::polymorphic_allocator<T>>>; // See P0887R1 for type_identity_t
}
pmr::vector pv({1, 2, 3}, mem_res); // OK with this definition
```

Finally, in the spirit of alias templates being simply an alias for the type, we do not propose allowing the programmer to write explicit deduction guides specifically for an alias template.

Algorithm

For deriving deduction guides for the alias templates from guides in the class, we use the following approach (for which we are very grateful for the invaluable assistance of Richard Smith):

1. Deduce template parameters for the deduction guide by deducing the right hand side of the deduction guide from the alias template. We do not require that this deduces all the template parameters as nondeducible contexts may of course occur in general
2. Substitute any deductions made back into the deduction guides. Since the previous step may not have deduced all template parameters of the deduction guide, the new guide may have template parameters from both the type alias and the original deduction guide.
3. Derive the corresponding deduction guide for the alias template by deducing the alias from the result type. Note that a constraint may be necessary as whether and how to deduce the alias from the result type may depend on the actual argument types.
4. The guide generated from the copy deduction guide should be given the precedence associated with copy deduction guides during overload resolution

Let us illustrate this process with an example. Consider the following example:

```
template<class T> using P = pair<int, T>;
```

Naively using the deduction guides from pair is not ideal because they cannot necessarily deduce objects of type `P` even from arguments that should obviously work, like `P({}, 0)`. However, let us apply the above procedure. The relevant deduction guide is

```
template<class A, class B> pair(A, B) -> pair<A, B>
```

Deducing `(A, B)` from `(int, T)` yield `int` for `A` and `T` for `B`. Now substitute back into the deduction guide to get a new deduction guide

```
template<class T> pair(int, T) -> pair<int, T>;
```

Deducing the template arguments for the alias template from this gives us the following deduction guide for the alias template

```
template<class T> P(int, T) -> P<T>;
```

A repository of additional expository materials and worked out examples used in the refinement of this algorithm is maintained online.

Deducing from inherited constructors

In C++17, deduction guides (implicit and explicit) are not inherited when constructors are inherited. According to the C++ Core Guidelines C.52, you should “use inheriting constructors to import constructors into a derived class that does not need further explicit initialization”. As the creator of such a thin wrapper has not asked in any way for the derived class to behave differently under construction, our experience is that users are surprised that construction behavior changes:

```
template <typename T> struct CallObserver requires Invocable<T> {
  CallObserver(T &&) : t(std::forward<T>(t)) {}
  virtual void observeCall() { t(); }
  T t;
};

template <typename T> struct CallLogger : public CallObserver<T> {
  using CallObserver<T>::CallObserver;
  virtual void observeCall() override { cout << "calling"; t();  }
};
```

C++17	Proposed
`CallObserver observer([]() { /* ... */ }); // OK`	`CallObserver observer([]() { /* ... */ }); // OK`
`CallLogger</*????*/> logger([]() { /* ... */ });`	`CallLogger logger([]() { /* ... */ }); // Now OK`

Note that inheriting the constructors of a base class must include inheriting all the deduction guides, not just the implicit ones. As a number of standard library writers use explicit guides to behave “as-if” their classes were defined as in the standard, such internal implementation details details would become visible if only the internal guides were inherited. We of course use the same algorithm for determining deduction guides for the base class template as described above for alias templates.

Wording

The wording for CTAD for aggregates has been included into the C++20 standard. An initial version of the wording can be found in P1816R0; substantial fixes have been done in P2082R1.

The wording for CTAD for alias templates has been included into the C++20 standard and can be found in P1814R0.

The wording for CTAD from inherited constructors was not finalized in time for C++20, but is currently being proposed for C++23 in P2582R0.